

Algoritmos de Aprendizado

- Regra de Hebb
- Perceptron
- Delta Rule (Least Mean Square)
- Back Propagation
- Hopfield
- Competitive Learning
- Radial Basis Functions (RBFs)

Delta Rule

• Características Básicas:

- Regra de Propagação → $net_j = \sum x_i \cdot w_{ji} + \theta_j$
- Função de Ativação → **Linear**: $s_j = k \cdot net_j$
- Topologia → **Uma única camada de processadores.**
- Algoritmo de Aprendizado → **Supervisionado**:
 $\Delta w_{ji} = \eta \cdot x_i \cdot (t_j - s_j)$
- Valores de Entrada/Saída → **Contínuos**

Delta Rule

- Também conhecida como:
 - Algoritmo Least Mean Square (LMS)
 - Regra de Widrow-Hoff
 - Regra de Adaline (Adaptive Linear Neuron)
- É uma generalização do **Perceptron**, estendendo a técnica para entradas e saídas contínuas

Delta Rule

Ao se apresentar um certo padrão p:

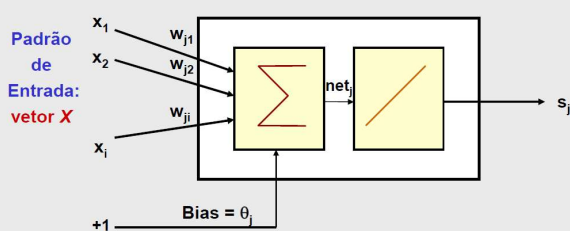
$$\Delta w_{ji} = \eta \cdot x_i \cdot (t_j - s_j)$$

Taxa de Aprendizado → $0 < \eta < 1$

Esta equação é obtida usando-se o método do **Gradiente Decrescente sobre o Erro Médio Quadrático** ("Least Mean Square Error")

Delta Rule

Modelo Básico do Neurônio Artificial:



Delta Rule

E_{SSE} → **Sum of Squared Errors**

$$E_{SSE} = \frac{1}{2} \sum_p \sum_j (t_{pj} - s_{pj})^2$$

- t_{pj} = valor desejado de saída do padrão p para o processador j da camada de saída
- s_{pj} = estado de ativação do processador j da camada de saída quando apresentado o padrão p

Delta Rule

Método do Gradiente Decrescente (GD)

$$\Delta w_{ji} = -\eta \frac{\delta E_j}{\delta w_{ji}}$$

Cada **peso sináptico i** do elemento processador **j** é atualizado proporcionalmente ao **negativo da derivada parcial do erro** deste processador com relação ao **peso sináptico i** .

Functional Link Network

Delta Rule

Logo:

$$\Delta w_{ji} = -\eta \frac{\delta E_j}{\delta w_{ji}} = -\eta \left(\frac{\delta E_j}{\delta s_j} \cdot \frac{\delta s_j}{\delta w_{ji}} \right)$$

$$E_j = \frac{1}{2} (t_j - s_j)^2 \quad s_j = k \sum x_i \cdot w_{ji} + \theta_j$$

$$\Delta w_{ji} = -\eta \cdot \left[2 \cdot \frac{1}{2} (t_j - s_j) \cdot (-1) \right] \cdot x_i$$

$$= -\eta \cdot [-(t_j - s_j)] \cdot x_i = \eta x_i (t_j - s_j)$$

Functional Link Network

- Aumenta o número de entradas artificialmente, tornando um problema **linearmente não-separável** tratável por uma rede de **uma única camada**.

Algoritmo de Aprendizado

- 1 Iniciar os pesos sinápticos com valores randômicos e pequenos ou **iguais a zero**;
- 2 Aplicar um novo padrão de entrada **p** , com seu respectivo valor desejado de saída (**t_j^p**), e verificar a saída da rede (**$s_j^p = k \sum x_i^p \cdot w_{ji}^p$**);
- 3 Calcular o erro na saída: **$e_j^p = t_j^p - s_j^p$**
- 4 Se **$e_j^p = 0$** , voltar ao passo 2;
Se **$e_j^p \neq 0$** , atualizar os pesos: **$\Delta w_{ji}^p = \eta \cdot x_i^p \cdot e_j^p$**
- 5 Voltar ao passo 2.

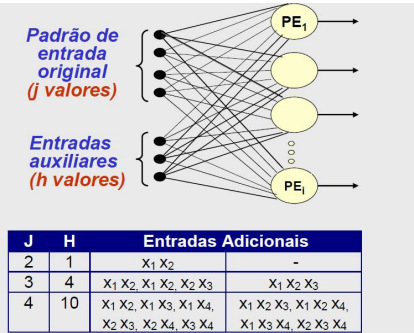
Functional Link Network

- Aumenta o número de entradas artificialmente, tornando um problema **linearmente não-separável** tratável por uma rede de **uma única camada**.



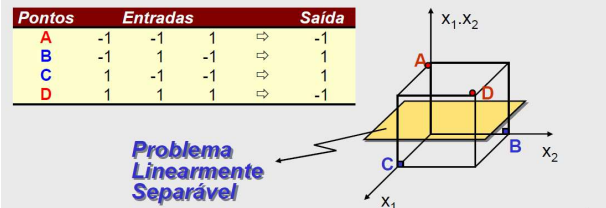
- O **ponto chave** é descobrir os valores destas novas entradas da rede!

Functional Link Network



Functional Link Network

- Exemplo do OU-EXCLUSIVO:
 - J = 2 (número de entradas originais - x_1, x_2)
 - H = 1 (número de entradas adicionais - $x_1.x_2$)



Fim do Delta Rule